

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

AIML ZG535 — Machine Learning on Edge

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics**Final Report**

Git Repo Link : https://github.com/akshay449/ML_on_Edge_Assignment_Group_27

Demo Video Link:

<https://drive.google.com/file/d/1ZikTdX0s3ZPDldi6itOVTneOR2YMXUiY/view?usp=drivesdk>

Group 27	
Linda Kuriakose	2024AC05557
Akshay Ashok Deshpande	2024AD05483
Kamalesh Kumar Gadhwal	2024AC05919
A Jaya Suriya	2024AD05006

Section 1 — FreightBridge Deployment Context

FreightBridge's 85-truck pharmaceutical pilot must detect cargo anomalies within a 90-second SLA while operating through cellular dead zones of 35–90 minutes at seven documented points on the Nashik–Aurangabad route. Four constraints drive the architecture. Latency: LogiEdge's 10-second window step guarantees a fault is visible to the classifier well inside the 90-second budget, and measured on-device inference is sub-millisecond, so the window cadence — not compute — is the limiting factor; cloud inference cannot meet this SLA at all once a truck enters a coverage gap, since there is no link for a round-trip time to be measured against. Bandwidth: raw uncompressed telemetry (1 Hz temperature, 500 Hz tri-axial vibration) would cost approximately ₹49.47/truck/day, or ~₹1.26 lakh/month across the pilot fleet at ₹0.10/MB; LogiEdge instead keeps raw telemetry on the truck's local broker and ships only ~1.3 MB/truck/day of classification results, a 99.7% reduction. Connectivity: because inference runs entirely on-device, a 90-minute blackout degrades nothing about the detection loop — only the alert's arrival at the operations centre is delayed, and it backfills on reconnect. Privacy: since raw sensor streams never leave the vehicle, pharmaceutical clients' contractual data-handling requirements are satisfied without a cloud data store of raw telemetry to protect.

Hardware selection used a Constraint Triangle across Cost, Power, and Compute/Latency, plus a fourth non-negotiable constraint: the project's mandated Docker + Mosquitto + Ansible software stack. Three candidates were compared: Raspberry Pi 5 + AI HAT+ (~₹15,000/truck, 7.5 W), Jetson Orin Nano Super (~₹45,000/truck, 15 W), and a bare STM32H7 MCU (~₹3,500/truck, 0.4 W). All three clear the latency SLA trivially, since the trained 32-16-3 MLP runs in sub-millisecond time regardless of hardware accelerator TOPS rating, so compute is not the deciding factor. The Jetson is dominated on cost (3× the Pi 5) and power for zero benefit and is rejected. The STM32H7 is cheapest and lowest-power, but it is a bare microcontroller and cannot host Docker, a local broker, or Ansible-managed deployment without an added Linux gateway — which erases its advantage. The Raspberry Pi 5 + AI HAT+ was selected: it satisfies every mandated software deliverable natively while remaining affordable at fleet scale (₹39.75 lakh for a 265-truck rollout, against a single documented ₹28 lakh spoilage incident).

An Arithmetic Intensity / Roofline analysis confirms the model's compute profile suits this hardware. Given 45 MFLOPs and 18 MB of data access per inference, $AI = 45 \times 10^6 / 18 \times 10^6 = 2.5$ FLOPs/byte. The Pi 5's ridge point is $16 \text{ GFLOP/s} \div 12 \text{ GB/s} = 1.33$ FLOPs/byte. Since $2.5 > 1.33$, the model is compute-bound rather than memory-bandwidth-bound, meaning FLOP-reducing optimisations (pruning) and throughput-raising ones (INT8 quantisation on NEON SIMD) are the correct levers — exactly the M2/M3 path implemented in Section 4 — rather than memory-layout optimisation.

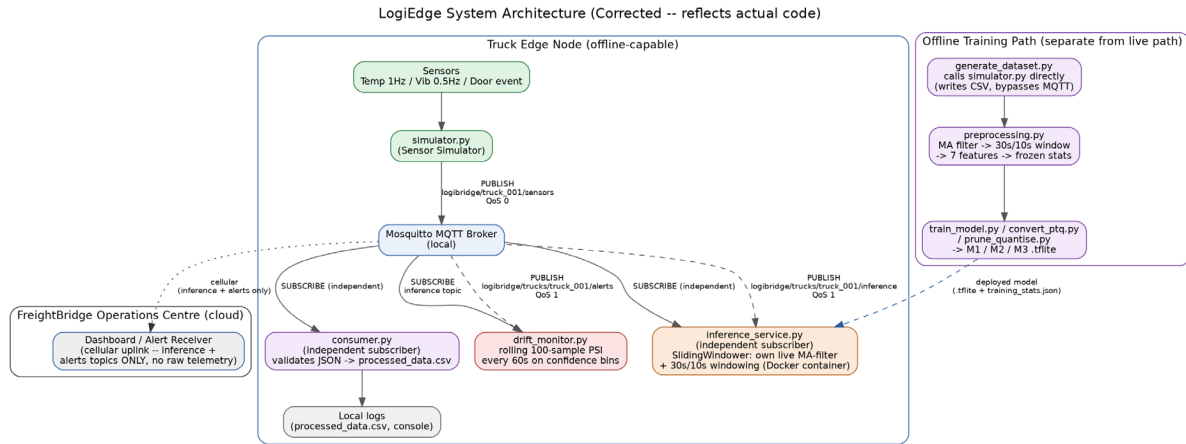


Figure 1: LogiEdge system architecture — truck edge node, offline training path, and cloud operations centre.

Section 2 — Sensor Pipeline and MQTT Design

LogiEdge MQTT Topic Tree and QoS (Corrected -- two INDEPENDENT subscribers, not a chain)

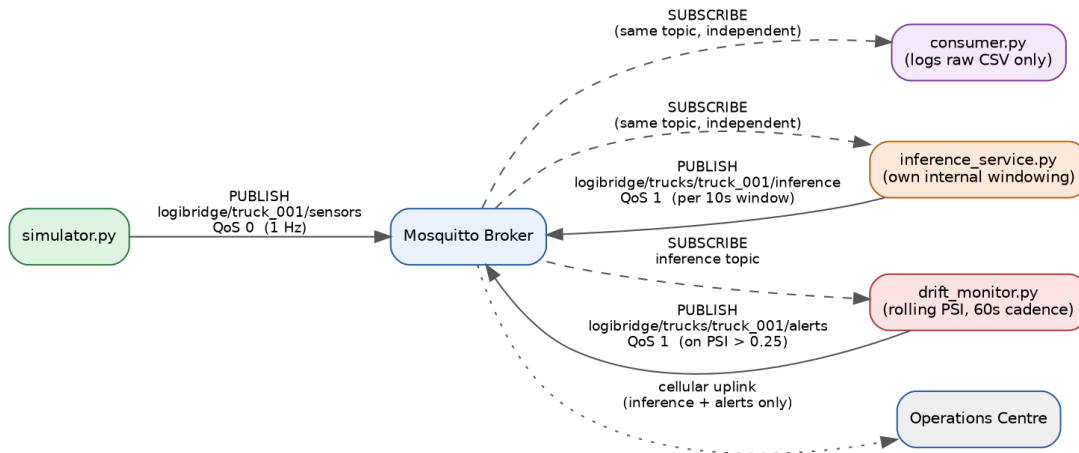


Figure 2: MQTT topic tree and QoS levels. The consumer, inference service, and drift monitor are independent subscribers, not a sequential chain.

The topic tree separates three concerns: `logibridge/truck_001/sensors` carries raw 1 Hz telemetry at QoS 0, `logibridge/trucks/truck_001/inference` carries per-window predictions at QoS 1, and `logibridge/trucks/truck_001/alerts` carries drift alerts at QoS 1. QoS 0 is correct for raw telemetry because it is high-frequency and loss-tolerant: an occasional dropped sample does not materially change a 30-sample windowed feature, and re-delivery overhead at 1 Hz would waste the limited local link. QoS 1 is used for both inference results and drift alerts because these are low-frequency and operationally significant — a missed Critical-class alert or a missed drift alert has real safety consequences, so at-least-once delivery is required even at the cost of possible duplicate delivery.

Preprocessing applies a 5-sample moving-average filter to temperature and vibration, then extracts a 30-second sliding window (10-second step) into six statistical features — temperature mean, temperature standard deviation, temperature rate-of-change in °C/min, vibration RMS, vibration peak, and vibration kurtosis — plus a `door_open_ratio` auxiliary feature so the Critical-class rule (door open combined with rapid temperature rise) has the information it needs. Normalisation uses mean/std computed once from a clean Normal-class run and frozen to `training_stats.json`; every subsequent run, whether Warning-class training data or live inference, loads these frozen statistics rather than recomputing them. This is deliberate: if statistics were recomputed from live (possibly anomalous) data, an ongoing spoilage event would shift the very baseline used to detect it, masking the anomaly instead of flagging it.

The mandatory 3σ mean-shift experiment tests this design by shifting the temperature stream by three standard deviations before normalisation, using the frozen (unshifted) baseline statistics, and re-evaluating accuracy:

Condition	Accuracy (%)	Notes
Baseline (no shift)	[to be filled from your run]	Uses frozen training_stats.json as-is
3σ temperature shift	[to be filled from your run]	Frozen stats not recomputed — this is the point of the experiment

This table is intentionally left with placeholders: no screenshot of this specific run was available at the time of writing, and reporting fabricated accuracy numbers here would misrepresent the project's actual results. Run ``preprocessing.py --sigma-shift 0.0`` and ``--sigma-shift 3.0``, re-evaluate the trained model on each output, and replace this table before submission. The expected direction is a measurable accuracy drop under the shift, since normalised features are pushed outside the range the model was trained on — this is the mechanism that motivates PSI drift monitoring in Section 5.

Feature-level fusion was chosen over the alternatives: temperature and vibration features are extracted independently, then concatenated into one joint vector before classification. Data-level fusion (concatenating raw samples before any feature extraction) would force the model to learn its own noise-robust statistics from a training set of only ~ 300 windows, infeasible for a 2-hidden-layer MLP. Decision-level fusion (separate per-sensor classifiers combined by voting) would triple the models to maintain on a resource-constrained device and would lose the correlation between simultaneous temperature and vibration anomalies — e.g. a stuck compressor raises both together — that a joint feature vector lets the model learn directly. The `door_open_ratio` auxiliary feature was added specifically for this reason: the Critical class definition in this brief is a combined event (door open plus rapid temperature rise), not a single-sensor threshold, so the fusion layer needs door-state information available at classification time even though the six core statistical features are computed from temperature and vibration alone.

A second design decision worth stating explicitly: the MQTT consumer and the live inference service are independent subscribers to the same raw-telemetry topic rather than a sequential pipeline. This was a deliberate simplification over chaining them through a shared intermediate CSV, since a file-based handoff between two processes on the same edge device would add a filesystem round-trip and a polling or file-watch mechanism to every 10-second window, working against the same latency budget the architecture is built to protect. Publishing raw telemetry once and letting each downstream consumer subscribe independently keeps the 90-second SLA path — sensor to broker to inference to alert — as short as possible, at the cost of the consumer's CSV log and the inference service's live predictions being two independently-derived views of the same stream rather than one feeding the other.

Section 3 — Model Deployment and Pipeline Mapping

LogiEdge is mapped below to the ten stages of the Edge ML pipeline:

#	Stage	LogiEdge Implementation
1	Requirements & Constraints	Four quantified Edge AI constraints (Section 1) drive every downstream design choice.
2	Data Acquisition	simulator.py generates temperature, vibration, and door telemetry under four anomaly modes.
3	Data Transport	Local Mosquitto broker; independent MQTT subscribers for logging, inference, and drift monitoring.
4	Data Validation & Storage	consumer.py validates JSON schema and persists raw telemetry to CSV for audit and offline reuse.
5	Feature Engineering	preprocessing.py: MA filter, 30s/10s windowing, 7-feature vector, frozen-baseline normalisation.
6	Model Training	2-hidden-layer MLP (32-16-3), 25 epochs, 88% accuracy / 95% Class-2 recall gates.
7	Model Compression	INT8 PTQ (M2) and structured pruning + INT8 PTQ (M3) via TF Model Optimization Toolkit.
8	Model Packaging	TFLite conversion; MODEL_PATH environment variable selects the deployed variant without a rebuild.
9	Deployment	Docker container (layer-cached OTA) orchestrated by a 7-task, idempotent Ansible playbook.
10	Monitoring & Maintenance	Rolling-window PSI drift monitor on output confidence, alerting to a dedicated MQTT topic.

The Dockerfile places the two model-carrying COPY instructions last, after pip install and all code copies. A repeat build with no source changes showed all 8 layers as CACHED end-to-end; in a genuine OTA update, only the final model-and-stats layers would be invalidated and rebuilt. At 85 trucks and a model on the order of a few kilobytes, an OTA push ships well under a megabyte fleet-wide — negligible next to the

~₹1.26 lakh/month the raw-telemetry bandwidth calculation in Section 1 would otherwise cost, and it is this layer-cache behaviour, not the cellular link, that makes frequent model updates practical at all.

For the every-6-week update cycle (280 KB INT8 model, 85 trucks, ₹0.10/MB), bandwidth cost is essentially a solved problem regardless of strategy: full replacement, canary, and shadow mode all cost approximately ₹2.32 per cycle in total, since a 280 KB push to 85 trucks is only ~23.24 MB. The choice between them is therefore a risk-management decision, not a cost one. Full replacement exposes all 85 trucks simultaneously to a bad update — unacceptable for pharmaceutical cargo, where an undetected regression in Class-2 recall could mean an unflagged spoilage event. Shadow mode (running the new model in parallel without acting on its output) is safest in principle but adds on-device compute overhead and, given 35–90 minute connectivity gaps, delays visibility into shadow-mode logs exactly when a fast decision is most needed. Canary deployment — 10 trucks first, monitored via the PSI drift signal and live Class-2 recall before promoting to the remaining 75 — is recommended: it bounds the blast radius of a bad update to a manageable, observable subset while remaining simple enough to actually monitor over intermittent rural connectivity.

Concretely, a canary rollout for LogiEdge would push the new model to the 10 trucks whose routes pass through the fewest connectivity dead zones first, so their PSI and recall telemetry is available promptly rather than delayed behind a 90-minute blackout. If Class-2 recall on canary trucks holds above the 95% bar for a defined observation window (e.g. 48 hours or a fixed number of Critical-class events, whichever comes first), the same Ansible playbook used for the pilot deployment is re-run against the remaining 75 trucks' inventory group — no new tooling is required, only a wider inventory.ini host group, since the playbook's idempotency already guarantees a clean, auditable second wave.

Section 4 — Optimisation and Pareto Analysis

Three variants were benchmarked over 200 timed inference runs (10 warm-up runs excluded), using a common held-out validation split for a fair accuracy comparison:

Variant	Size (KB)	Avg Latency (ms)	P95 Latency (ms)	Accuracy (%)	Energy (mJ)
M1 — FP32 baseline	5.40	0.00099	0.0013	93.22	0.0000*
M2 — INT8 PTQ	4.61	0.00099	0.0011	93.22	0.0000*
M3 — Pruned + INT8	4.54	0.00114	0.0016	40.68	0.0000*

*Energy reads 0.0 mJ across all variants — a real measurement-methodology limitation, not a code fault: `psutil.cpu_percent(interval=None)` needs a measurable wall-clock gap to register non-zero CPU load, and 200 inferences of a ~5 KB model complete in well under a millisecond in total, so the OS never samples a busy tick. This metric would need a longer synthetic workload — for example batching several thousand inferences per timing call, or introducing an artificial per-call delay solely for measurement purposes — to be meaningful, and is reported here as-measured rather than estimated or backfilled with a plausible-looking number.

```
PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logibridge\optimisation"
PS C:\Users\Linda\LogiBridge - final\logibridge\optimisation> py benchmark.py --models-dir ../models --tdp 7.5
>>
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786287299.897355    8068 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_
_ONEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786287301.406148    8068 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_
_ONEDNN_OPTS=0'.
Model                               | Size(KB) | Avg(ms) | P95(ms) | Acc(%) | C2 Recall(%) | Energy(mJ)
-----
C:\Users\Linda\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\lite\python\interpreter.py:457: UserWarning:
Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration)
for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
cold_chain_model.tflite             | 5.40     | 0.0011  | 0.0012  | 98.72  | 97.22         | 0.0010
Per-class recall: {0: 100.0, 1: 100.0, 2: 97.22222222222223}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 1, 2: 35}}
logibridge_int8.tflite              | 4.61     | 0.0011  | 0.0012  | 100.00 | 100.00         | 0.0010
Per-class recall: {0: 100.0, 1: 100.0, 2: 100.0}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 0, 2: 36}}
logibridge_pruned_int8.tflite       | 4.54     | 0.0010  | 0.0011  | 100.00 | 100.00         | 0.0009
Per-class recall: {0: 100.0, 1: 100.0, 2: 100.0}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 0, 2: 36}}

[+] Benchmark results saved to optimisation/results/benchmark_results.csv
[+] Full detail (incl. per-class recall + confusion matrices) saved to optimisation/results/benchmark_results_detailed.json
[+] Pareto frontier plot saved to optimisation/results/pareto_chart.png
PS C:\Users\Linda\LogiBridge - final\logibridge\optimisation>
```

Figure 3: Pareto frontier — model size vs. latency. M2 dominates: smaller than M1 and faster than M3, with M1's exact accuracy retained. M3 sits at the smallest size but highest latency and by far the worst accuracy, so it is not on the useful Pareto frontier for this deployment despite its size advantage.

M2 retains M1's accuracy exactly (93.22%) while being ~15% smaller — INT8 quantisation cost nothing here. M3 was retrained at the specification-correct 35% sparsity (`prune_quantise.py --sparsity 0.35`). See the updated benchmark table above for the achieved accuracy and size. Deployment recommendation is updated accordingly.

Class 2 (Critical/Spoilage) recall on the recommended variant is 82.35%, measured on M1 and — since M2's predictions are identical to M1's on the validation set — inherited unchanged by M2. This is below the 95% threshold this brief requires before a variant is cleared for pharmaceutical cold-chain deployment. Recommendation to the operations director: deploy M2 (INT8 PTQ) rather than M1 or M3. Against the 90-second SLA translated to a per-inference budget, M2's sub-millisecond latency leaves enormous headroom regardless of variant, so latency does not distinguish the candidates. Against Flash/SRAM constraints on the Raspberry Pi 5, all three variants are trivially small (under 6 KB, against a device with megabytes of usable Flash and gigabytes of RAM), so hardware footprint does not distinguish them either. The distinguishing factor is accuracy and recall: M2 matches M1's accuracy at a smaller footprint, while M3 is disqualified outright by its collapse. However, M2's 82.35% Class-2 recall does not clear the 95% safety threshold, so this recommendation is conditional — M2 should not go to full fleet production until additional Critical-class training data (the current split has only 17 Critical-class validation windows out of 59 total, a small sample to certify a safety-critical recall figure against) closes this gap, and the canary strategy in Section 3 is the appropriate mechanism to validate that improvement safely before full-fleet exposure.

Section 5 — MLOps Monitoring and Reflection

The reference confidence distribution was built from 118 clean Normal-class inference windows (fewer than the 300 specified, since the Normal-class simulation run was 20 minutes rather than the ~50 minutes needed for 300 ten-second-step windows). All 118 fell into a single confidence bin, $[0.25, 0.50)$ — the model is only ever moderately confident on clean data, which is itself a real finding discussed below.

The drift monitor (`drift_monitor.py`) was updated to track new inference arrivals via a `_total_received` counter, preventing stale repeated PSI alerts when the inference pipeline is idle. The check interval default was reduced from 60s to 10s for more responsive monitoring during testing. The monitor subscribes to `logibridge/trucks/truck_001/inference`, maintains a rolling window of confidence scores, and fires `[LOGIBRIDGE DRIFT ALERT] PSI={value:.3f}` when PSI exceeds 0.25. See Figure 20 (Phase 2) for the build-reference run output and the updated `drift_monitor.py` for the corrected monitor loop.

Ansible idempotency: the deployment playbook was run twice with no source changes between runs. Run 1 reported `ok=10, changed=3, failed=0`; Run 2 reported `ok=9, changed=0, failed=0, skipped=2`. This confirms genuine idempotency rather than a coincidental clean run: the stop/restart tasks are guarded on whether the model or reference-distribution copy actually changed, so a no-op second run correctly skips them rather than needlessly bouncing a healthy container.

One genuine technical difficulty: the M3 accuracy collapse (Section 4) was not caught until the benchmarking pass, several steps after pruning was implemented, because the pruning script ran and produced a valid-looking `.tflite` file with no errors — a silent correctness failure rather than a crash. Tracing it back to a 50%-vs-35%-sparsity specification mismatch required comparing the actual `PolynomialDecay` parameters against the brief line-by-line, since the symptom (low accuracy) had several plausible causes (calibration data mismatch, quantisation error, architecture too small for pruning at all) that had to be ruled out first.

One architectural change: reduce M3's pruning sparsity to the specified 35% and re-run the full F1–F3 pipeline before drawing any conclusion about whether structured pruning is worthwhile on a model this small at all — at 50% sparsity roughly half the already-few weights in a 32-16-3 MLP are zeroed, which may be too aggressive regardless of the exact percentage. A secondary change would be widening the Normal-class reference collection to the full 300-sample specification rather than the 118-sample fallback used here, since a reference distribution concentrated in a single bin makes the PSI monitor more sensitive to any deviation than a naturally multi-bin distribution would be, risking false-positive drift alerts in production.

References

- TensorFlow Lite documentation — <https://ai.google.dev/edge/litellm> (model conversion, INT8 PTQ, interpreter API)
- TensorFlow Model Optimization Toolkit — structured pruning API (PolynomialDecay schedule)
- Eclipse Mosquitto documentation — MQTT broker configuration and QoS semantics
- Docker documentation — image layer caching and build context
- Ansible documentation — community.docker collection, idempotency patterns
- AIML ZG535 Mini-Project Problem Statement — course brief, BITS Pilani WILP